

中文语义镜像语言 MVP

3099404236^{1*}
独立原型项目

本项目提出一种面向 AI 生成代码审阅的中文语义镜像语言：它不是把英文关键字逐字翻译成中文，而是把程序中需要人判断的语义选择投影为严格、直观、可执行的中文表示。首个最小可行产品解析一段整数序列筛选与排序程序，构造统一语义模型，再分别交给 Python 解释器和 C++17/GCC 后端执行，自动比较退出状态、标准输出和标准错误。7 项自动测试以及正常、边界和错误输入实验表明，这条“中文程序可运行、目标代码可生成、两条路径可判定同对同错”的链路已经成立。当前结果只证明有限子集的工程可行性，不代表已经支持任意 C++。

中文编程语言 | AI 代码审阅 | 语义投影 | 同对同错 | 可执行原型
<https://github.com/3099404236>

初衷：AI 时代的困难正在改变

AI 已经可以快速生成大量代码。新的瓶颈不再只是“人能不能把每个符号敲出来”，而是人能不能在有限时间内检查生成结果有没有选错算法、边界、数据关系、所有权或副作用。现有语言通常使用英文标识，并包含大量历史语法和细节规则。它们适合精确执行，却不一定适合非专业读者快速审阅。

本项目由一个直接的想法开始：如果 AI 写完代码后，还能给出一个中文、直观、可以真正运行的版本，人就能把注意力放在更重要的判断上。这里的中文版本不能只是解释性注释，因为注释可能与代码分离；它必须参与执行和验证，目标程序出错时，中文版本也应尽量在对应语义处出错。

核心问题

能否构造一种严格的中文表示，让人主要审阅“做了什么、选择了什么”，同时让 AI 和编译器处理大量机械实现细节？

这不是主张基础知识完全不重要。相反，人仍需要理解数据是否共享、边界是否包含、对象何时消失、一次操作是否修改原数据等基本机制。变化在于学习和审阅的重点：少背孤立格式，多检查会改变程序行为的选择。

设计目标与边界

“直观”不能变成每次运行都按个人喜好随意解释。一个可执行语言仍需要固定规范，否则同一句话无法稳定复现。个性化通过 fork 解决：任何人都可以复制语法和语义定义，形成自己的正式版本；同一版本内部仍保持严格一致。

本项目当前也明确不做三件事：不宣称已经替代 C++，不把关键词翻译等同于语义镜像，不把有限测试结果当作形式化等价证明。

MVP 的统一语义链路

首个版本只选取一条足够小、但完整的任务：读取整数序列和阈值，按比较条件筛选，排序并输出。用户编写的中文程序如下：

```
任务 筛选并排序
输入 整数序列 数据
输入 整数 阈值
选择 结果 = 数据 中 严格大于 阈值的数字
排列 结果 从小到大
输出 结果
结束
```

系统不是分别维护两套互不相关的实现，而是先解析出一个统一的 Program 语义对象。这个对象记录输入名称、比较运算、排序方向、输出对象和原始中文行号。两条执行路径都从它出发：

这种结构把“中文如何写”和“机器最终如何执行”分开。将来增加后端时，不必让每种目标语言重新解释自由中

目标	含义	v0.1 状态
中文	关键程序语义用中文表达，降低阅读门槛。	已实现
直观	优先展示算法、条件、数据流和副作用。	部分实现
严格	不是自由自然语言，每条语句都有固定语法。	已实现
可执行	中文版本自己可以得到结果和错误。	已实现
同对同错	中文路径与目标路径比较退出码、输出和错误。	有限子集已实现
兼容现有语言	以多后端生成 C++、Rust 等目标程序。	仅 C++17 原型
可 fork	规范稳定，但任何人可复制并发展新方言。	工程约定

表 1 项目目标以及首个版本的实际覆盖范围。

严格中文程序	→	统一语义模型	→	Python 中文解释器
同一中文来源	→	同一 Program	→	C++17 生成器 + GCC
两条执行结果	→	退出码 / 输出 / 错误	→	自动一致性判定

表 2 v0.1 的最小语义镜像链路。

场景	中文解释器	C++17/GCC	判定
正常输入	8 8 12	8 8 12	同对
边界输入	6 9 10	6 9 10	同对
数量不足	输入数量错误	同一输入数量错误	同错
非整数	要求 32 位整数	同一错误	同错
中文语法错误	报告中文行号	不生成目标程序	前端阻止

表 3 当前版本的代表性验证场景。

文；它们读取同一个语义模型即可。中文语法错误也在进入目标代码生成前被阻止，并报告原始行号。

“同对同错”具体比较什么

正常结果一致还不够。如果中文解释器接受了一个输入，而生成的 C++ 拒绝它，用户仍然会被误导。因此原型为两条路径统一了输入约束和错误文本，并比较三个观察量：

1. 退出码：成功为 0，输入错误为 2，超时等异常另行区分。
2. 标准输出：正常计算结果必须逐字符一致。
3. 标准错误：失败原因必须一致，不能一边成功、一边报错。

自动测试共 7 项，覆盖解析摘要、中文执行、C++ 源码行号映射、真实 GCC 对照、输入数量错误、非整数错误和中文语法定位。网页端另外手工检查了正常输入、边界输入、错误输入和窄屏布局。

人与 AI 的分工

这个语言的目标不是隐藏所有细节，而是把细节按责任重新分层。

例如，“严格大于”与“大于或等于”只有一个符号差别，却会改变边界结果；这类选择必须显式保留给人检查。相反，如何遍历容器、如何拼接输出、编译命令包含哪些机械参数，可以由生成器稳定完成。

实现与复现

原型由四部分组成：mirror_compiler.py 负责解析、解释、生成 C++ 和结果比较；server.py 提供本机 HTTP 接口；web/ 提供可编辑演示界面；test_mirror_compiler.py 提供自动测试。运行环境只要求 Python 3.10+ 和支持 C++17 的 g++。

命令行复现：

```
python -m unittest discover -s code -p "test_*.py"
-v
python code\mirror_compiler.py data\example.mirror
--input-file data\example-input.txt --compare
```

第二条命令会输出一致性判定和结果 8 8 12。网页演示通过 python code\server.py 启动，并在浏览器访问 <http://127.0.0.1:8787>。

局限性与下一步

当前版本最大的限制是语言范围极窄。它还没有变量声明、通用表达式、循环、函数、结构体、类、指针、异常或模块系统，也没有处理 C++ 中未定义行为、浮点差异、并发和外部 I/O 等难题。“同对同错”目前是对可观察结果的工程比较，并非形式化证明。

下一步应按风险逐层扩展，而不是一次支持全部 C++：

1. 增加整数变量、条件、循环和可视化语义树。
2. 明确“修改原对象”与“产生新对象”，把副作用写进中文语法。

层次	AI / 工具主要负责	人主要检查
机械实现	语法展开、类型适配、样板代码、编译和测试。	是否出现无法解释的隐藏行为。
语义选择	把明确选择映射到多个后端。	算法、比较条件、排序方向、修改或复制。
资源关系	生成释放、错误处理和边界检查代码。	共享还是独占、生命周期、外部副作用。
验证	运行双路径、构造测试、定位不一致。	测试是否覆盖真正担心的失败方式。

表 4 建议的人机责任边界。

3. 引入数组、记录和所有权描述，验证深拷贝、共享与生命周期。
4. 为每个语义功能同时增加解释器测试、C++ 对照测试和错误映射。
5. 定义版本化规范，使不同 fork 能声明兼容范围和转换边界。

判断

这个 MVP 已经证明“中文严格语法 → 统一语义 → 两条可执行路径 → 同对同错判定”可以完整跑通。它尚未证明完整语言可行，但已经把问题从想法推进为可扩展、可测试的工程对象。

面向人的语义审阅清单

当 AI 生成的目标代码越来越长时，中文镜像不应复述每一个括号，而应保证下面这些会改变行为的问题能够被直接看见：

1. 输入从哪里来，允许哪些值，边界是否包含。
2. 输出到哪里，失败时返回什么，是否产生外部效果。
3. 操作修改原对象，还是创建并返回一个新对象。
4. 两个名字代表独立数据，还是共享同一块数据。
5. 数据由谁拥有，什么时候释放，离开函数后是否继续存在。
6. 循环和递归在什么条件下停止，能否重复进入同一状态。
7. 排序、筛选、搜索等步骤的先后顺序是否会改变结果。
8. 发生错误时，中文行号能否映射到目标代码和测试证据。

这份清单也决定语言后续扩展的优先级。只有能够回答这些问题的新语法才值得加入；纯粹为了模仿目标语言表面的复杂形式，不是本项目的目标。

固定规范与个人 fork

不同人的“直观”并不完全相同，但运行中的语言不能临时猜测使用者的意思。因此本项目区分两个层次：

- **版本内部固定**：每个关键句、类型和错误都有稳定定义，程序和测试可以重复运行。
- **版本之间可修改**：使用者可以 fork 语法表、语义模型和错误表达，形成自己的正式方言。

未来的兼容性不应只写“支持本语言”，而应声明具体版本和语义能力。例如，一个 fork 可以改变中文用词，但仍复用同一个 Program 模型；另一个 fork 若改变了“选择”的副作用，就必须使用新的语义版本，并提供转换或明确拒绝转换。这样既允许语言适应人，也避免“同一句话每个人各自理解”造成不可执行的混乱。

结论

AI 时代不应要求每个人都先适应一套历史悠久但不一定直观的表示，才能检查机器生成的程序。更现实的方向，是让系统同时提供适合执行的目标代码和适合人审阅的语义投影，并通过共同语义模型约束二者。中文语义镜像语言 v0.1 只迈出了很小的一步，但它已经具备三个关键性质：中文版本能够运行，目标 C++ 能够生成，二者能够自动检查同对同错。

这使后续讨论有了可验证的基础：哪些细节应隐藏，哪些选择必须显式，哪些错误必须映射，以及一门面向个人直观、又保持严格规范的语言应当如何演进。